

Streams in Java

↳ Data processing

Employee

↳ sal > 50,000
↳ name
↳ sort

$(e_1, e_2, \dots, e_n)$

↓

(e_1, e_5, e_7)

↓

(n_1, n_5, n_7)

↓

n_5, n_1, n_7

```
class Employee {  
    int empId;  
    String name;  
    ...  
}
```

employees

→ List<String> result = new ArrayList<>();

```
for (Employee e : employees) {  
    if (e.getSalary() > 50000) {  
        result.add(e.getName());  
    }  
}
```

Collections.sort(result);

→ List<String> result = new ArrayList<>();

```
for (Employee e : employees) {  
    if (e.getSalary() > 50000) {  
        result.add(e.getName());  
    }  
}
```

Collections.sort(result);

employees

List<String> result

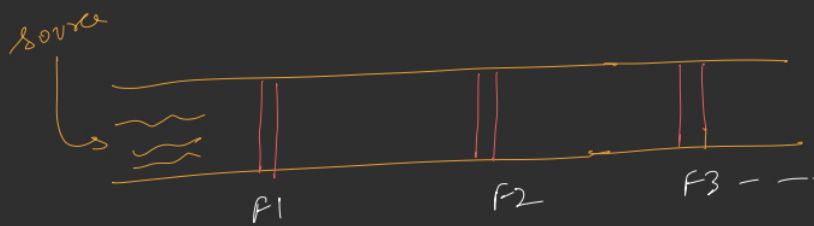
↑ = employees.stream()

- filter(x → x > 50000)
- map(Employee::getName)
- sorted()
- toList();

Streams

↳ A stream is a tool for processing a sequence of data through a chain of operations.

Water Stream (water pipeline)



Java Streams

`numbers.stream()`

- `filter(x → x > 10)`
- `filter(x → x % 2 == 0)`
- `map(x → x * x)`
- `forEach(System.out::println);`

Streams → collecting data (X)

↳ Data process (✓)

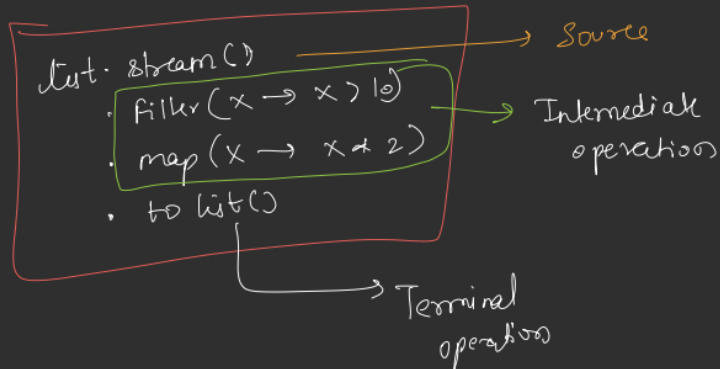
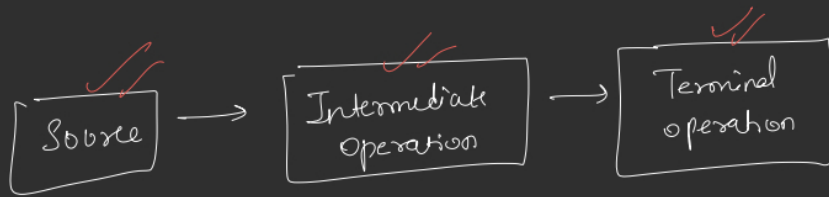
numbers

7	11	23	43	71	10
---	----	----	----	----	----

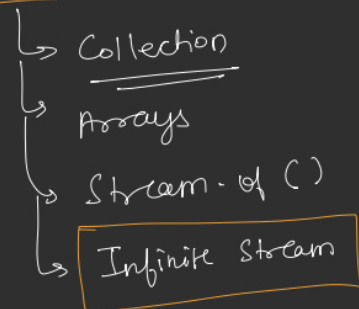
↳ ArrayList

`numbers.stream()`

Stream pipeline Architecture (list = [1, 3, 7, 5, 9])

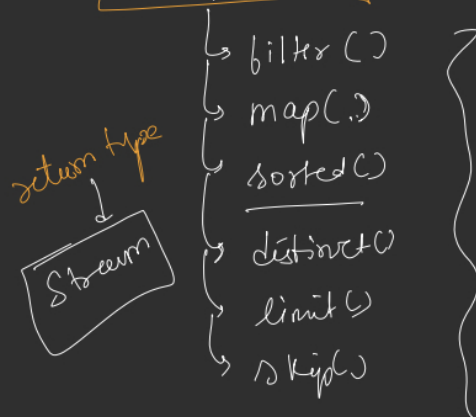


Source



list.stream()

Intermediate operation



list.stream()

- filter(x -> x > 10)
- map()

Terminal operation

- ↳ forEach()
- ↳ toList()
- ↳ collect()
- ↳ count()
- ↳ findFirst()

No Terminal operation



No Stream executed



Lazy loading

list = [5, 12, 7, 20]

```
list.stream()
    .filter(x -> x > 10)
    .map(x -> x * 2)
    .toList() findFirst()
```

Vertical processing

result = [24, 40]

Lazy Evaluation



Short Circuiting

Stream<Integer> s = list.stream()

s.forEach(System.out::println);

s.forEach() (X)

Stream Sources

- ↳ Collection
- ↳ Array
- ↳ Stream.of()
- ↳ Infinite Stream
- ↳ Primitive Streams

list.stream()

cos

list.parallelStream()

Array

↳ `String[] arr = {"Aditya", "Rohit", "--"};`

`Stream<String> s = Arrays.stream(arr)`

Stream.of(...)

↳

`Stream<Integer> s =`

`Stream.of(1, 2, 5, 7, 9);`

EmptyStream

↳ `Stream.empty()`

1, 2, 3, 4, - - - - -

Infinite Stream

- ↳ `iterate()`
- ↳ `generate()`

`Stream.iterate((seed), nextFn)`

`Stream.iterate(1, x → x+1)`

• `limit(10)`

• `forEach(System.out::println);`

`iterate()` → depends on
prev value

`generate` → No
dependence

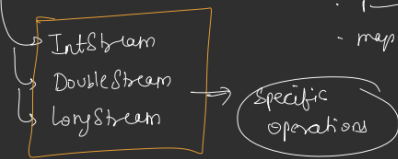
`Stream.generate(Supplier)`

↓
`Stream.generate(Math::random);`

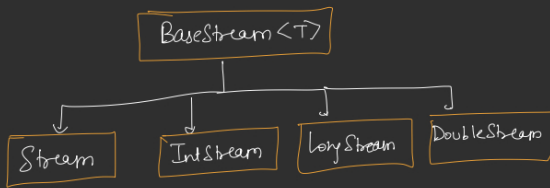
Primitive Stream

`Stream<Integer> s = list.stream()`

- `filter(x → x > 10)`
- `flatMap(x → x.collect(toList()))`
- `map(x → x * 2)`



`IntStream s = IntStream.of(1, 2, 3, ...)`



`IntStream s = IntStream.of(1, 2, 3);`
`LongStream s2 = s.mapToLong(x → x);`
`DoubleStream s3 = s.mapToDouble(x → x);`

Object → primitive

`Stream<Integer> s = list.stream();`

`IntStream s2 = s.mapToInt();`

Primitive → Object

`IntStream s = IntStream.of(1, 2, 3);`

`Stream<Integer> s2 = s.boxed();`